1. Prepare Multiple Nodes on Alibaba Cloud

1.1. Purchase Single Node on Alibaba Cloud

Step 1: Click https://homenew.console.aliyun.com/home/dashboard/ProductAndService, choose the ECS service,

Step 2: Create a new instance, choose pay as you go, choose ecs.c7.2xlarge, choose Ubuntu 16.04 64 bits, choose ESSD, choose no IPv4 address, choose your password Y0urpassword,

Step 3: Double-check the order,

Step 4: Pay the order,

Step 5: Access the ECS instance via Alibaba Workbench, where the default username is root,

So far, the ECS instance cannot access the Internet because it has no IP address.

1.2. Purchase a NAT Gateway

Step 1: Click https://vpc.console.aliyun.com/nat/cn-guangzhou/nats, purchase 2 public IP addresses, create a NAT gateway,

Step 2: After purchasing 2 public IP addresses, we have 8.163.56.8 and 8.138.164.210,

1.3. Purchase Multiple Node on Alibaba Cloud

Step 1: Click https://vpc.console.aliyun.com/vpc/cn-guangzhou/vpcs, purchase another 10 ECS nodes,

choose pay as you go, choose ecs.c7.2xlarge, choose Ubuntu 16.04 64 bits, choose ESSD, choose no IPv4 address, choose your password Y0urpassword,

1.4. Configure the NAT Gateway

Step 1: Click https://vpc.console.aliyun.com/nat/cn-guangzhou/nats, configure the NAT gateway, where the IP address 8.163.56.8 for DNAT and 8.138.164.210 for SNAT (DNAT for Internet users and SNAT for VM inside). Currently, we have 1 NAT, 2 Public IP addresses, and 11 ECS, our target is as follows:

For NAT (Public IP):

SNAT	DNAT
8.138.164.210	8.163.56.8

For ECS maps (Inside IP):

SNAT	DNAT
-	172.31.210.67
	172.31.210.70
	172.31.210.71
	172.31.210.72
	172.31.210.73
	172.31.210.74
	172.31.210.75
	172.31.210.76
	172.31.210.77
	172.31.210.78
	172.31.210.79

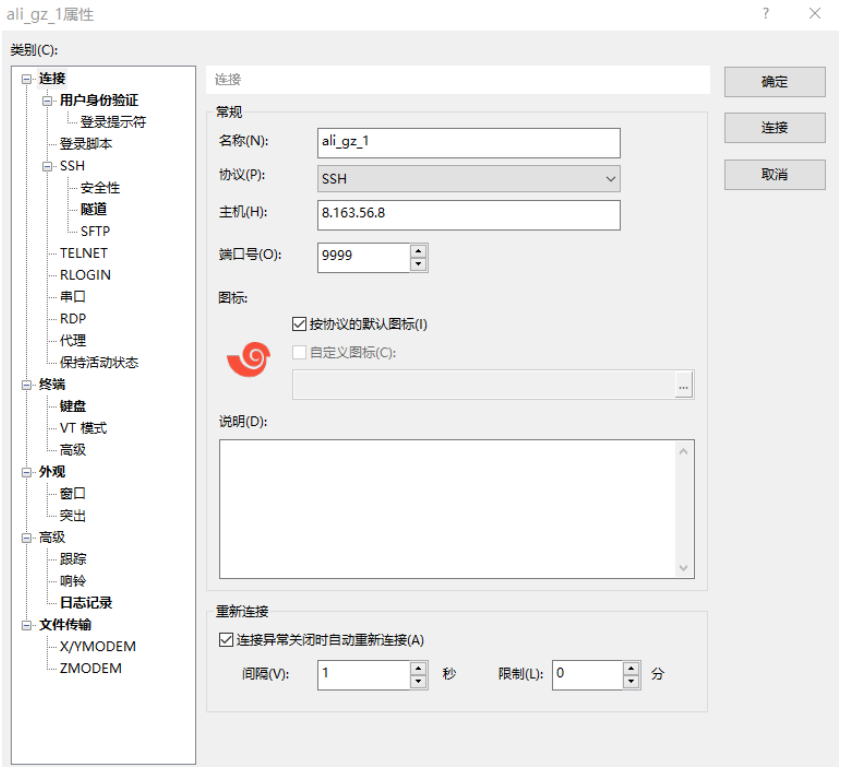
Step 2: Configure the SNAT,

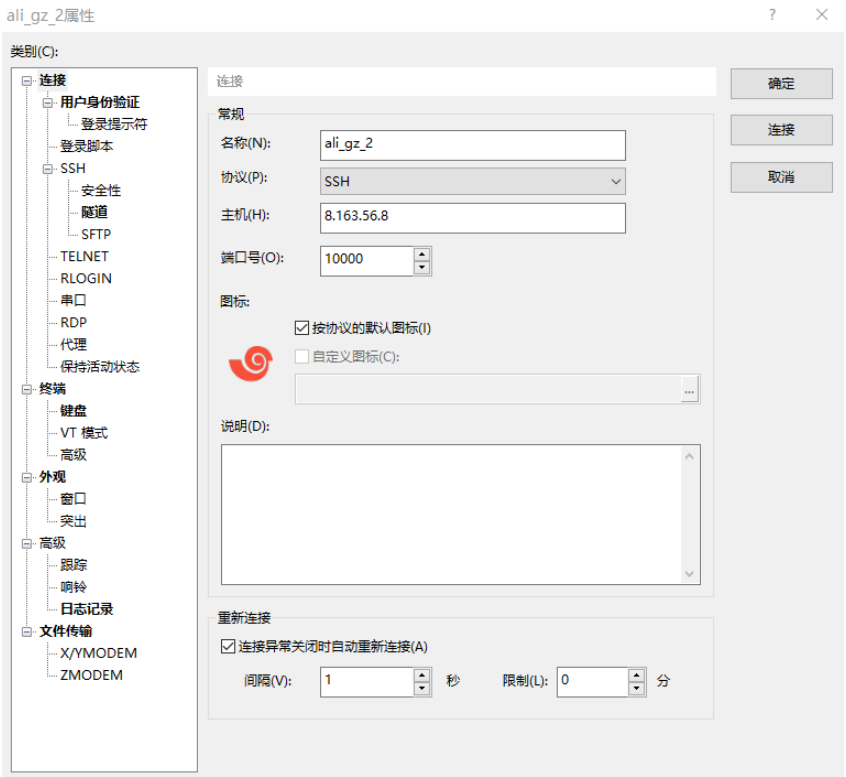
Step 3: Configure the DNAT,

ECS maps:

DNAT Public IP and Port Number	DNAT Inside IP
8.163.56.8:9999	172.31.210.67
8.163.56.8:10000	172.31.210.70
8.163.56.8:10001	172.31.210.71
8.163.56.8:10002	172.31.210.72
8.163.56.8:10003	172.31.210.73
8.163.56.8:10004	172.31.210.74
8.163.56.8:10005	172.31.210.75
8.163.56.8:10006	172.31.210.76
8.163.56.8:10007	172.31.210.77
8.163.56.8:10008	172.31.210.78
8.163.56.8:10009	172.31.210.79

Access the inside IPs via Xshell,





1.5. Configure the password-free SSH

Step 1: The target is as follows,

Host with free-password SSH to	
172.31.210.67	-
	172.31.210.70
	172.31.210.71
	172.31.210.72
	172.31.210.73
	172.31.210.74
	172.31.210.75
	172.31.210.76
	172.31.210.77
	172.31.210.78
	172.31.210.79

Step 2: Configure the 172.31.210.67,

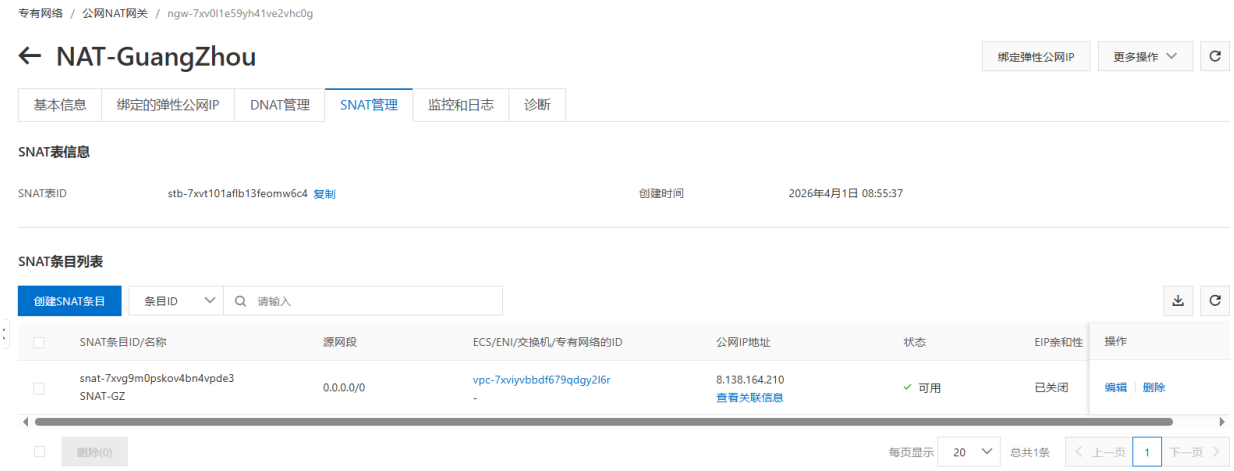
```
$ ssh-keygen

$ ssh root@172.31.210.70 'mkdir -p .ssh && cat > .ssh/authorized_keys' < ~/.ssh/id_rsa.pub
$ ssh root@172.31.210.71 'mkdir -p .ssh && cat > .ssh/authorized_keys' < ~/.ssh/id_rsa.pub
$ ssh root@172.31.210.72 'mkdir -p .ssh && cat > .ssh/authorized_keys' < ~/.ssh/id_rsa.pub
$ ssh root@172.31.210.73 'mkdir -p .ssh && cat > .ssh/authorized_keys' < ~/.ssh/id_rsa.pub
$ ssh root@172.31.210.74 'mkdir -p .ssh && cat > .ssh/authorized_keys' < ~/.ssh/id_rsa.pub
$ ssh root@172.31.210.75 'mkdir -p .ssh && cat > .ssh/authorized_keys' < ~/.ssh/id_rsa.pub
$ ssh root@172.31.210.76 'mkdir -p .ssh && cat > .ssh/authorized_keys' < ~/.ssh/id_rsa.pub
$ ssh root@172.31.210.77 'mkdir -p .ssh && cat > .ssh/authorized_keys' < ~/.ssh/id_rsa.pub
$ ssh root@172.31.210.78 'mkdir -p .ssh && cat > .ssh/authorized_keys' < ~/.ssh/id_rsa.pub
$ ssh root@172.31.210.79 'mkdir -p .ssh && cat > .ssh/authorized_keys' < ~/.ssh/id_rsa.pub

$ ssh 172.31.210.70 "echo 'hi'"
$ ssh 172.31.210.71 "echo 'hi'"
$ ssh 172.31.210.72 "echo 'hi'"
$ ssh 172.31.210.73 "echo 'hi'"
$ ssh 172.31.210.74 "echo 'hi'"
$ ssh 172.31.210.75 "echo 'hi'"
$ ssh 172.31.210.76 "echo 'hi'"
$ ssh 172.31.210.77 "echo 'hi'"
$ ssh 172.31.210.78 "echo 'hi'"
$ ssh 172.31.210.79 "echo 'hi'"
```

1.6. Free the NAT Gateway

Step 1: Click https://vpc.console.aliyun.com/nat/cn-guangzhou/nats/ngw-7xv0l1e59yh41ve2vhc0g/snats, delete SNAT,



Step 2: Click https://vpc.console.aliyun.com/nat/cn-guangzhou/nats, unbind public IP, free public IP,

Step 3: Click https://homenew.console.aliyun.com/home/dashboard/ProductAndService, delete ECS,

Step 4: Click https://vpc.console.aliyun.com/nat/cn-guangzhou/nats, delete NAT

2. Run Single Node Fabric Test-Network

2.1. Configure Each Node

Step 1: Configure the host names,

```
# sudo vi /etc/hosts
:set paste
I
```

```
172.31.210.67 peer0.org1.example.com

172.31.210.70 client1.peer
172.31.210.71 client2.peer
172.31.210.72 client3.peer
172.31.210.73 client4.peer
172.31.210.74 client5.peer
172.31.210.75 client6.peer
172.31.210.76 client7.peer

172.31.210.77 order.example.com
172.31.210.78 order2.example.com
172.31.210.79 order3.example.com
```

2.2. Install Dependencies on Each Node

Step 1: Install the prerequisites on each node,

```
// Prerequisite: Git
$ sudo apt-get update
$ sudo apt purge -y git
$ sudo apt-get install -y git // =1:2.7.4-0ubuntu1.9
$ git version // 2.7.4
$ export PATH=$PATH:/usr/lib/git-core
$ echo 'export PATH=$PATH:/usr/lib/git-core' >> ~/.bashrc
$ source ~/.bashrc
```

Step 2: Install the cURL on each node,

```
// Prerequisite: cURL
$ sudo apt-get install -y curl
$ curl –version // 7.47.0
```

Step 3: Install the wget on each node,

```
// Prerequisite: wget
```

```
$ sudo apt-get install -y wget=1.17.1-1ubuntu1.5
```

Step 4: Install Docker on each node,

```
// Prerequisite: Docker 17.06.2-ce or greater: Docker 17.12.0-ce
$ sudo apt-get purge -y docker-ce docker-ce-cli
$ wget https://download.docker.com/linux/ubuntu/gpg
$ sudo apt-key add gpg
$ echo 'deb [arch=amd64] https://download.docker.com/linux/ubuntu xenial stable' | sudo tee
/etc/apt/sources.list.d/docker.list
$ sudo apt-get update
$ apt-cache madison docker-ce
$ sudo apt-get install docker-ce=17.12.0~ce-0~ubuntu
$ sudo usermod -aG docker $USER
$ docker version
```

Step 5 (if outside the Great Wall): Install Docker Compose on each node,

```
// Prerequisite: Docker Compose 1.14.0 or greater: 1.21.0
$ sudo apt-get purge -y docker-compose
$ sudo curl -L https://github.com/docker/compose/releases/download/1.21.0/docker-compose-
`uname -s`-`uname -m` -o /usr/local/bin/docker-compose
$ sudo chmod +x /usr/local/bin/docker-compose
$ docker-compose version
```

Step 5 (if within the Great Wall): Install Docker Compose on each node,

```
// Prerequisite: Docker Compose 1.14.0 or greater: 1.21.0
$ sudo apt-get purge -y docker-compose
$ mkdir downloads
$ cd $HOME/downloads
$ wget https://gitee.com/joe32065/fabric_2_2/releases/download/fabric-v2.2.0/docker-compose
$ sudo chmod +x docker-compose
$ sudo mv docker-compose /usr/local/bin
$ docker-compose version
```

Step 6 (if outside the Great Wall): Install Golang on each node,

```
// Prerequisite: Go v1.14.8
$ cd $HOME/downloads
$ wget https://golang.org/dl/go1.14.8.linux-amd64.tar.gz
$ tar -xzf go1.14.8.linux-amd64.tar.gz
$ sudo rm -rf /usr/local/go
$ sudo mv go /usr/local
$ mkdir -p $HOME/go
$ echo 'export PATH=$PATH:/usr/local/go/bin' >> ~/.bashrc
```

```
$ echo 'export GOPATH=$HOME/go' >> ~/.bashrc
$ source ~/.bashrc
$ go version // go1.14.8 linux/amd64
```

Step 6 (if within the Great Wall): Install the Golang on each node,

```
// Prerequisite: Go v1.14.8
$ cd $HOME/downloads
$ ... // download go1.14.8.linux-amd64.tar.gz on node one, e.g., via winscp,
$ scp go1.14.8.linux-amd64.tar.gz root@172.31.210.70:$HOME/downloads
$ scp go1.14.8.linux-amd64.tar.gz root@172.31.210.71:$HOME/downloads
$ ...
$ scp go1.14.8.linux-amd64.tar.gz root@172.31.210.79:$HOME/downloads

$ tar -xzf go1.14.8.linux-amd64.tar.gz
$ sudo mv go /usr/local
$ mkdir -p $HOME/go
$ echo 'export PATH=$PATH:/usr/local/go/bin' >> ~/.bashrc
$ echo 'export GOPATH=$HOME/go' >> ~/.bashrc
$ source ~/.bashrc
$ go version // go1.14.8 linux/amd64
```

Step 7: Install Node.js on each node,

```
// Prerequisite: Node.js v12.20.1 or higher
$ cd $HOME/downloads
$ curl -sL https://deb.nodesource.com/setup_12.x -o nodesource_setup.sh
$ sudo chmod +x nodesource_setup.sh
$ sudo bash nodesource_setup.sh
$ sudo apt-get install -y nodejs --allow-unauthenticated
$ nodejs -v // v12.22.12

$ curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.35.3/install.sh | bash
$ // close the terminals, and open the terminals again
$ cd $HOME/downloads
$ sudo npm -g install nvm
$ nvm ls available
$ nvm install 12.20.1
$ nvm use 12.20.1
```

Step 8: Install the NTP on each node,

```
$ sudo apt-get install ntp
$ sudo service ntp start
```

2.3. Install Fabric 2.2.0

Step 1 (if outside the Great Wall): Install Fabric 2.2.0, including Fabric Samples on each node,

```
$ cd $HOME
$ sudo rm -rf fabric-samples
$ curl -sSL https://bit.ly/2ysbOFE | bash -s -- 2.2.0
```

Step 1 (if within the Great Wall): Install Fabric 2.2.0, including Fabric Samples on each node,

```
$ cd $HOME
$ sudo rm -rf fabric-samples
$ sudo apt-get install unzip
$ cd $HOME
$ wget https://gitee.com/joe32065/fabric_2_2/releases/download/fabric-v2.2.0/fabric-v2.2.0.zip
$ unzip fabric-v2.2.0.zip
$ cd fabric-v2.2.0/scripts
```

In bootstrap.sh -> ”pullBinaries()”,

```
download "${BINARY_FILE}"
"https://github.com/hyperledger/fabric/releases/download/v${VERSION}/${BINARY_FILE}"
download "${CA_BINARY_FILE}" "https://github.com/hyperledger/fabric-
ca/releases/download/v${CA_VERSION}/${CA_BINARY_FILE}"
```

Replace with the two lines,

```
download "${BINARY_FILE}"
"https://gitee.com/joe320653/fabric/attach_files/842230/download/${BINARY_FILE}"
download "${CA_BINARY_FILE}" "https://gitee.com/joe320653/fabric-
ca/attach_files/842289/download/${CA_BINARY_FILE}"
```

In bootstrap.sh -> ” cloneSamplesRepo()”,

```
git clone -b master https://github.com/hyperledger/fabric-samples.git && cd fabric-samples && git
checkout v${VERSION}
```

Replace with the two lines,

```
git clone -b master https://gitee.com/joe320653/fabric-samples.git && cd fabric-samples && git
checkout v${VERSION}
```

Step 2: Install docker images and fabric-samples on each node,

```
$ cd $HOME/fabric-v2.2.0/scripts
$ chmod 777 *.sh

$ // configure the docker container service within the Great Wall
$ cd $HOME
$ sudo mkdir -p /etc/docker
$ sudo tee /etc/docker/daemon.json <<-'EOF'
{
  "registry-mirrors": ["https://2htzxp9d.mirror.aliyuncs.com"]
}
EOF
$ sudo systemctl daemon-reload
$ sudo systemctl restart docker

$ cd $HOME/fabric-v2.2.0/scripts
$ sudo ./bootstrap.sh
$ mv fabric-samples $HOME
```

2.4. Run Test-Network

Step 1 (if outside the Great Wall): Run test network on each node,

```
$ cd $HOME/fabric-samples/test-network
$ ./network.sh up
$ ./network.sh createChannel
$ ./network.sh deployCC -ccn basic -ccp ../asset-transfer-basic/chaincode-go -ccl go
$ ./network.sh down
```

The success information is as follows,

```
Using organization 1
Querying chaincode definition on peer0.org1 on channel 'mychannel'...
Attempting to Query committed status on peer0.org1, Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2
MSP: true]
Query chaincode definition successful on peer0.org1 on channel 'mychannel'
Using organization 2
Querying chaincode definition on peer0.org2 on channel 'mychannel'...
Attempting to Query committed status on peer0.org2, Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2
MSP: true]
Query chaincode definition successful on peer0.org2 on channel 'mychannel'
Chaincode initialization is not required
```

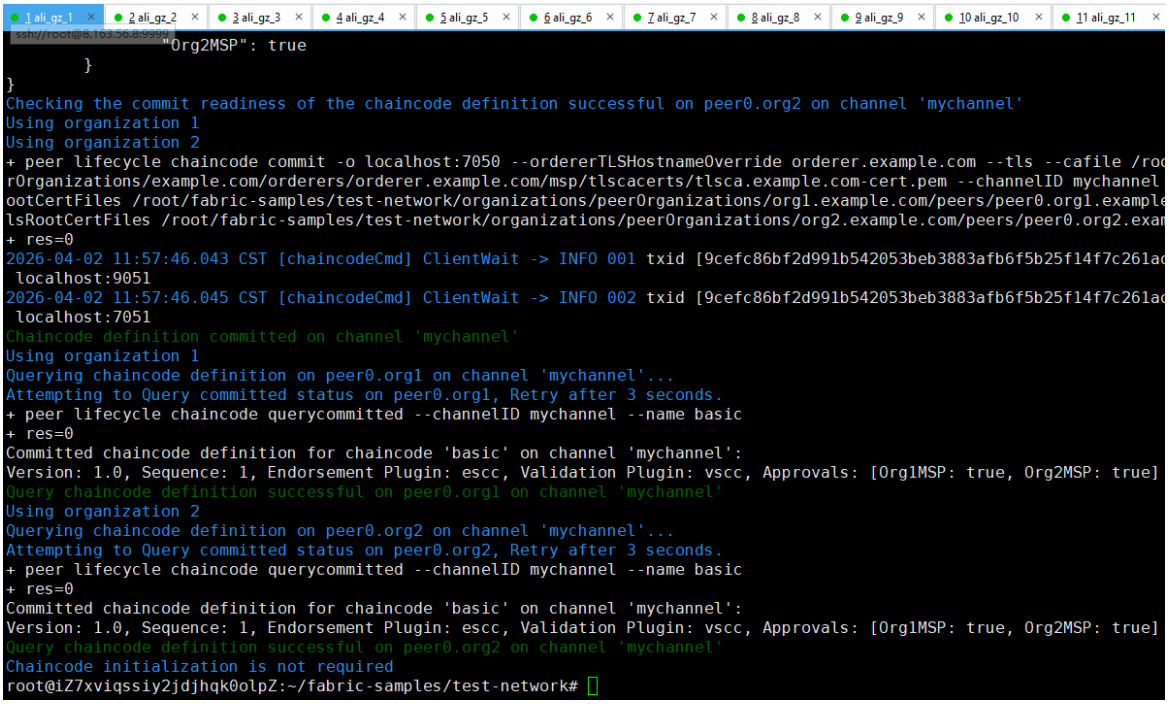
Step 1 (if within the Great Wall): Run test network on each node,

Set the Golang Proxy,

```
$ go env -w GOPROXY=https://goproxy.cn
```

Run the test network,

```
$ cd $HOME/fabric-samples/test-network
$ ./network.sh up
$ ./network.sh createChannel
$ ./network.sh deployCC -ccn basic -ccp ../asset-transfer-basic/chaincode-go -ccl go
$ ./network.sh down
```



3. Run Multiple Nodes on Fabric 2.2.0 (1 CLI + 2 Peers + 1 Orderer)

3.1. Prepare the Hosts

Step 1 (Prepare): All machines prepare the host names,

```
$ vi /etc/hosts
```

```
# 172.31.210.67 cli (of peer0.org1)
172.31.210.70 peer0.org1.example.com
172.31.210.71 peer0.org2.example.com
172.31.210.72 orderer.example.com
```

Step 2 (Prepare): CLI prepares the Test-Network files,

```
$ cd $HOME/fabric-samples
$ mkdir demo-first
$ cd $HOME/fabric-samples/demo-first
$ vi crypto-config.yaml
$ vi configtx.yaml

$ vi peer0-org1.yaml
$ vi peer0-org2.yaml

$ vi orderer1.yaml

$ vi cli.yaml // (of peer0.org1)

$ vi $HOME/fabric-samples/chaincode/abstore/go/abstore.go // our proposed key-value chaincode
$ vi $HOME/fabric-samples/chaincode/abstore/go/go.mod
```

Step 2 (Prepare): All machines prepare the timestamp alignment,

```
$ date +%s // check the timestamp

$ sudo apt-get install ntpdate // align the timestamp
$ sudo ntpdate cn.pool.ntp.org
$ sudo hwclock –systohc

$ date +%s // check the timestamp again
```

Step 3 (Run): CLI runs the initial block generation,

```
$ cd $HOME/fabric-samples/demo-first
$ chmod +x *.sh
$ ./config.sh
```

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# ./config.sh
org1.example.com
org2.example.com
2026-04-14 16:21:05.926 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:05.942 CST [common.tools.configtxgen.localconfig] completeInitialization -> INFO 002 orderer type: etcdraft
2026-04-14 16:21:05.942 CST [common.tools.configtxgen.localconfig] completeInitialization -> INFO 003 Orderer.EtcdRaft.Options unset, setting to tick_interval:"500ms" election_
ck:10 heartbeat_tick:1 max_inflight_blocks:5 snapshot_interval_size:16777216
2026-04-14 16:21:05.942 CST [common.tools.configtxgen.localconfig] Load -> INFO 004 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:05.943 CST [common.tools.configtxgen] doOutputBlock -> INFO 005 Generating genesis block
2026-04-14 16:21:05.943 CST [common.tools.configtxgen] doOutputBlock -> INFO 006 Writing genesis block
2026-04-14 16:21:05.957 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:05.973 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:05.973 CST [common.tools.configtxgen] doOutputChannelCreateTx -> INFO 003 Generating new channel configtx
2026-04-14 16:21:05.974 CST [common.tools.configtxgen] doOutputChannelCreateTx -> INFO 004 Writing new channel tx
2026-04-14 16:21:05.988 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:06.003 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:06.003 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 003 Generating anchor peer update
2026-04-14 16:21:06.004 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 004 Writing anchor peer update
2026-04-14 16:21:06.018 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:06.034 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:06.034 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 003 Generating anchor peer update
2026-04-14 16:21:06.034 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 004 Writing anchor peer update
```

```
$ cd $HOME/fabric-samples/demo-first/workload/tls
$ chmod +x *.sh

$ cd $HOME/fabric-samples/demo-first/workload
$ chmod +x *.sh
$ wget https://gitee.com/joe32065/fabric_2_2/releases/download/fabric-v2.2.0/node_modules.zip
$ unzip node_modules.zip

$ cd $HOME/fabric-samples/demo-first/scripts
$ chmod +x *.sh
```

Step 4 (Run): CLI scp the contents to all machines including peers and orderers,

```
$ cd $HOME/fabric-samples/demo-first
$ ./scp.sh
```

```
priv_sk                                   100%  241    0.2KB/s   00:0
tlsca.example.com-cert.pem                100%  847    0.8KB/s   00:0
orderer.example.com-cert.pem              100%  769    0.8KB/s   00:0
tlsca.example.com-cert.pem                100%  847    0.8KB/s   00:0
Admin@example.com-cert.pem                100%  769    0.8KB/s   00:0
ca.example.com-cert.pem                   100%  839    0.8KB/s   00:0
priv_sk                                   100%  241    0.2KB/s   00:0
ca.crt                                    100%  847    0.8KB/s   00:0
server.key                                100%  241    0.2KB/s   00:0
server.crt                                100%  916    0.9KB/s   00:0
Admin@example.com-cert.pem                100%  769    0.8KB/s   00:0
tlsca.example.com-cert.pem                100%  847    0.8KB/s   00:0
Admin@example.com-cert.pem                100%  769    0.8KB/s   00:0
ca.example.com-cert.pem                   100%  839    0.8KB/s   00:0
priv_sk                                   100%  241    0.2KB/s   00:0
ca.crt                                    100%  847    0.8KB/s   00:0
client.crt                                100%  814    0.8KB/s   00:0
client.key                                100%  241    0.2KB/s   00:0
priv_sk                                   100%  241    0.2KB/s   00:0
ca.example.com-cert.pem                   100%  839    0.8KB/s   00:0
```

Step 5 (Run): Orderer1 sets up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first
$ ./orderer1.sh
```

```
root@iZ7xviqssiy2jdsfcldpc2Z:~/fabric-samples/demo-first# ./orderer1.sh
Creating volume "net_orderer.example.com" with default driver
Creating orderer.example.com ... done
```

```
$ docker ps
```

```
root@iZ7xviqssiy2jdsfcldpc2Z:~/fabric-samples/demo-first# docker ps
CONTAINER ID        IMAGE                               COMMAND             CREATED             STATUS              PORTS                    NAMES
4a207f175833        hyperledger/fabric-orderer:2.2.0    "orderer"           8 seconds ago       Up 8 seconds        0.0.0.0:7050->7050/tcp   orderer.example.com
```

Step 6 (Run): Peer1 sets up the Docker container,

$ cd $HOME/fabric-samples/demo-first
$ ./**peer1.sh**

```
root@iZ7xviqssiy2jdsfcldpbvZ:~/fabric-samples/demo-first# ./peer1.sh
Creating volume "net_peer0.org1.example.com" with default driver
Creating peer0.org1.example.com ... done
```

$ docker ps

```
root@iZ7xviqssiy2jdsfcldpbvZ:~/fabric-samples/demo-first# docker ps
CONTAINER ID        IMAGE                               COMMAND             CREATED             STATUS              PORTS                    NAMES
38fb1129b2ae        hyperledger/fabric-peer:2.2.0       "peer node start"   2 seconds ago       Up 1 second         0.0.0.0:7051->7051/tcp   peer0.org1.example.com
```

Step 7 (Run): Peer2 setups the docker container,

$ cd $HOME/fabric-samples/demo-first
$ ./**peer2.sh**

```
root@iZ7xviqssiy2jdsfcldpc1Z:~/fabric-samples/demo-first# ./peer2.sh
Creating volume "net_peer0.org2.example.com" with default driver
Creating peer0.org2.example.com ... done
```

$ docker ps

```
root@iZ7xviqssiy2jdsfcldpc1Z:~/fabric-samples/demo-first# docker ps
CONTAINER ID        IMAGE                               COMMAND             CREATED             STATUS              PORTS                                        NAMES
e88f655db1d9        hyperledger/fabric-peer:2.2.0       "peer node start"   2 seconds ago       Up 1 second         7051/tcp, 0.0.0.0:8051->8051/tcp             peer0.org2.example.com
```

Step 8 (Run): CLI setups the docker container,

$ cd $HOME/fabric-samples/demo-first
$ ./**cli.sh**

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# ./cli.sh
Creating cli ... done
```

$ docker ps

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# docker ps
CONTAINER ID        IMAGE                               COMMAND             CREATED             STATUS              PORTS               NAMES
30e6fbad4c51        hyperledger/fabric-tools:2.2.0      "/bin/bash"         2 seconds ago       Up 1 second                             cli
```

3.2. Start the Network

Step 1 (Run): CLI (of peer0.org1) connects to Peer0.org1.example.com:7051, creates a channel.block, and then all peers join the channel.block,

// CLI goes to `peer0.org1.example.com:7051`
$ docker exec -it cli bash

// Environment Configuration for `peer0.org1.example.com:7051`
cd scripts
./**step1.sh** // All peers join the channel

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# docker ps
CONTAINER ID        IMAGE                          COMMAND             CREATED             STATUS              PORTS               NAMES
30e6fbad4c51        hyperledger/fabric-tools:2.2.0   "/bin/bash"         2 seconds ago       Up 1 second                             cli
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# docker exec -it cli bash
bash-5.0# cd scripts
bash-5.0# ./step1.sh
2026-04-14 08:26:37.630 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:37.647 UTC [cli.common] readBlock -> INFO 002 Expect block, but got status: &{NOT_FOUND}
2026-04-14 08:26:37.649 UTC [channelCmd] InitCmdFactory -> INFO 003 Endorser and orderer connections initialized
2026-04-14 08:26:37.850 UTC [cli.common] readBlock -> INFO 004 Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-14 08:26:37.852 UTC [channelCmd] InitCmdFactory -> INFO 005 Endorser and orderer connections initialized
2026-04-14 08:26:38.053 UTC [cli.common] readBlock -> INFO 006 Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-14 08:26:38.054 UTC [channelCmd] InitCmdFactory -> INFO 007 Endorser and orderer connections initialized
2026-04-14 08:26:38.255 UTC [cli.common] readBlock -> INFO 008 Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-14 08:26:38.257 UTC [channelCmd] InitCmdFactory -> INFO 009 Endorser and orderer connections initialized
2026-04-14 08:26:38.457 UTC [cli.common] readBlock -> INFO 00a Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-14 08:26:38.459 UTC [channelCmd] InitCmdFactory -> INFO 00b Endorser and orderer connections initialized
2026-04-14 08:26:38.659 UTC [cli.common] readBlock -> INFO 00c Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-14 08:26:38.661 UTC [channelCmd] InitCmdFactory -> INFO 00d Endorser and orderer connections initialized
2026-04-14 08:26:38.864 UTC [cli.common] readBlock -> INFO 00e Received block: 0
2026-04-14 08:26:38.897 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:38.921 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join channel
2026-04-14 08:26:38.953 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:38.977 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join channel
2026-04-14 08:26:39.004 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:39.015 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update
2026-04-14 08:26:39.040 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:39.050 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update
bash-5.0#
```

./**step2.sh** // All peers join the chaincode

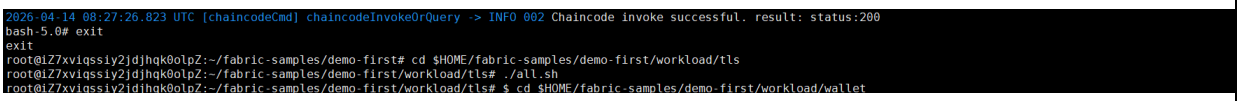
```
11ac10e3978491cdaab7d56219165c0b45\022\006mycc_1" >
2026-04-14 08:27:14.093 UTC [cli.lifecycle.chaincode] submitInstallProposal -> INFO 002 Chaincode code package identifier: mycc_1:9c2d699dc0bb37ea77cfbadd75670011ac10e3978491cda
b7d56219165c0b45
2026-04-14 08:27:20.092 UTC [cli.lifecycle.chaincode] submitInstallProposal -> INFO 001 Installed remotely: response:<status:200 payload:"\nGmycc_1:9c2d699dc0bb37ea77cfbadd7567(
11ac10e3978491cdaab7d56219165c0b45\022\006mycc_1" >
2026-04-14 08:27:20.093 UTC [cli.lifecycle.chaincode] submitInstallProposal -> INFO 002 Chaincode code package identifier: mycc_1:9c2d699dc0bb37ea77cfbadd75670011ac10e3978491cda
b7d56219165c0b45
2026-04-14 08:27:20.160 UTC [cli.lifecycle.chaincode] setOrdererClient -> INFO 001 Retrieved channel (mychannel) orderer endpoint: orderer.example.com:7050
2026-04-14 08:27:21.182 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [2e49555a3a8dd90810de8e5e90973f90a615a62711c3be05cc97ddf7b1cc0a23] committed with status (VALID) at
2026-04-14 08:27:21.218 UTC [cli.lifecycle.chaincode] setOrdererClient -> INFO 001 Retrieved channel (mychannel) orderer endpoint: orderer.example.com:7050
2026-04-14 08:27:22.238 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [a720bb2d81a5e1dc2f56ab369b6df082355423c38337e372ffc9f80a515d6f26] committed with status (VALID) at
{
        "approvals": {
                "Org1MSP": true,
                "Org2MSP": true
        }
}
2026-04-14 08:27:23.402 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [f8eba34bdcfcd726a6959a47dc60c95e05cf7652a8b7f4cf80c7b44f70bd236b] committed with status (VALID) at peer0
rg1.example.com:7051
2026-04-14 08:27:23.402 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [f8eba34bdcfcd726a6959a47dc60c95e05cf7652a8b7f4cf80c7b44f70bd236b] committed with status (VALID) at peer0
rg2.example.com:8051
2026-04-14 08:27:24.693 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [862b6b71fe7bdc81df4b23af39e63d1e1017b2c3b39e34b6cdf6ac6213bb905c] committed with status (VALID) at peer0
rg1.example.com:7051
2026-04-14 08:27:24.693 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
2026-04-14 08:27:25.741 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [1722e9e0ffb97b289c38f53946f397b242b6d3ff591f2ba4f48ae6ed239311f3] committed with status (VALID) at peer0
rg1.example.com:7051
2026-04-14 08:27:25.742 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
200tps
2026-04-14 08:27:26.823 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [64c2861c9c16c9c93e549c87eab9181e87ff23827fb1207ae4c213c802b92110] committed with status (VALID) at peer0
rg2.example.com:8051
2026-04-14 08:27:26.823 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
bash-5.0#
```

3.4. Start Nodejs SDK

Step 1 (Run): CLI configures the Node.js SDK folder and tls, wallet, and workload files for all peers,

```
# exit
$ mkdir -p $HOME/fabric-samples/demo-first/workload
$ mkdir -p $HOME/fabric-samples/demo-first/workload/tls
$ mkdir -p $HOME/fabric-samples/demo-first/workload/wallet
$ mkdir -p $HOME/fabric-samples/demo-first/workload/result

$ cd $HOME/fabric-samples/demo-first/workload/tls
$ vi read_pem.py
$ vi prepare_tls_peer0_org1.sh
$ vi prepare_tls_peer0_org2.sh
$ ./all.sh
```




```
$ cd $HOME/fabric-samples/demo-first/workload/wallet
$ vi prepare_wallet_user1_org1.py
$ vi prepare_wallet_user1_org2.py
$ ./all.sh

$ cd $HOME/fabric-samples/demo-first/workload
$ mkdir sdk-peer0-org1
$ mkdir sdk-peer0-org2
$ vi invoke.js
$ vi prepare_sdk_peer0_org1.sh
$ vi prepare_sdk_peer0_org2.sh
$ ./all.sh
```

Step 2 (Prepare): CLI prepares the Nodejs SDK dependencies on CLI. Note that you may pass this step by copying the compiled nodejs source code https://gitee.com/joe32065/fabric_2_2/releases/download/fabric-v2.2.0/node_modules.zip,

```
$ cd $HOME/fabric-samples/demo-first/workload
$ vi package.json
$ npm install
$ vi $HOME/fabric-samples/demo-first/workload/node_modules/fabric-network/lib/transaction.js
```

Step 3 (Run): Cli prepares wallets of User1@org1.example.com and User1@org1.example.com,

```
$ cd $HOME/fabric-samples/demo-first/workload/wallet
$ rm -rf *.id
$ python prepare_wallet_user1_org1.py
```

$ python prepare_wallet_user1_org2.py
$ ./**all.sh**

Step 4 (Run): CLI invokes and queries transactions via Nodejs,

$ cd $HOME/fabric-samples/demo-first/workload
$ ./**scp.sh** // CLI copies files to all machines

$ ./**ssh.sh** **> result/benchlog**
$ ls result -l

```
root@iZ7xviqssiy2jdsfcldpbvZ:~# cd $HOME/fabric-samples/demo-first/workload/sdk-peer0-org1
root@iZ7xviqssiy2jdsfcldpbvZ:~/fabric-samples/demo-first/workload/sdk-peer0-org1# nodejs invoke.js 60 100 10 1
Wallet path: /root/fabric-samples/demo-first/workload/wallet
Send_Endorsing_Proposal: 1776155562.442
Send_Endorsing_Proposal: 1776155562.447
Send_Endorsing_Proposal: 1776155562.45
Send_Endorsing_Proposal: 1776155562.452
Send_Endorsing_Proposal: 1776155562.455
Send_Endorsing_Proposal: 1776155562.458
Send_Endorsing_Proposal: 1776155562.46
Send_Endorsing_Proposal: 1776155562.462
Send_Endorsing_Proposal: 1776155562.464
Send_Endorsing_Proposal: 1776155562.468
Send_Endorsing_Proposal: 1776155562.471
Send_Endorsing_Proposal: 1776155562.474
Send_Endorsing_Proposal: 1776155562.476
Send_Endorsing_Proposal: 1776155562.478
Send_Endorsing_Proposal: 1776155562.48
Send_Endorsing_Proposal: 1776155562.482
Send_Endorsing_Proposal: 1776155562.484
Send_Endorsing_Proposal: 1776155562.487
Send_Endorsing_Proposal: 1776155562.488
Send_Endorsing_Proposal: 1776155562.49
Send_Endorsing_Proposal: 1776155562.492
Send_Endorsing_Proposal: 1776155562.494
Send_Endorsing_Proposal: 1776155562.495
Send_Endorsing_Proposal: 1776155562.497
Send_Endorsing_Proposal: 1776155562.499
Send_Endorsing_Proposal: 1776155562.502
Send_Endorsing_Proposal: 1776155562.504
Send_Endorsing_Proposal: 1776155562.507
```

// $ node query.js
// $ node invoke.js 30 500 20 1

Step 5 (Run): Analyze the log,

// CLI analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ python **readthroughput_p1.py** // rate of sending endorsed txs

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first/workload/result# python readthroughput_p1.py
Avg_Send_Rate_Phrase_1:
5.7471256178
```

$ sudo apt-get install python-numpy
$ python readdelay_p1.py // delay of endorsing phase

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first/workload/result# python readdelay_p1.py
Mean_Delay_Endorse:
0.0188799798489
Mean_Delay_Orderer:
0.00146238137317
Mean_Delay_Commit:
0.178739959002
CDF with the probability of 50% 0.00100016593933
CDF with the probability of 90% 0.00200009346008
CDF with the probability of 95% 0.00300002098083
```

// Orderer analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ sudo apt-get install python-numpy
//$ docker logs containername >& ordererlog
$ **docker logs $(docker ps -a | grep 'orderer'| awk '{print $1 }') >& ordererlog**
// $ scp ordererlog chwang@1xx.1xx.7xx.1xx:/home/chwang
$ python readthroughput_p2.py // rate of generating blocks

```
root@iZ7xviqssiy2jdsfcldpc2Z:~/fabric-samples/demo-first/workload/result# python readthroughput_p2.py
Block_ID:
lastPersistedBlock=[24]

lastPersistedBlock=[25]

lastPersistedBlock=[26]

lastPersistedBlock=[27]

lastPersistedBlock=[28]

lastPersistedBlock=[29]

lastPersistedBlock=[30]

lastPersistedBlock=[31]

lastPersistedBlock=[32]
```

// Peer analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ sudo apt-get install python-numpy
//$ docker logs containername >& peerlog
$ **docker logs $(docker ps -a | grep 'peer node start'| awk '{print $1 }') >& peerlog**
// $ scp peerlog chwang@1xx.1xx.7xx.1xx:/home/chwang
$ python2 readthroughput_p3.py

```
root@iZ7xviqssiy2jdsfcldpbvZ:~/fabric-samples/demo-first/workload/result# python2 readthroughput_p3.py
block id:
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
```

// Get block size
peer channel fetch 228 last.block -o orderer.example.com:7050 -c mychannel --tls --cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem

Step 6 (Run): All nodes run the clean procedure,

$ cd $HOME/fabric-samples/demo-first
$ ./clean.sh